Payment Failure Analysis in SQL

Requirements & Analysis Document

Oct 26, 2026 · Prepared by @PRATIK MISHRA

1. Introduction

This document captures the business analysis behind a self-directed SQL analytics exercise on digital payment failures. It simulates a UPI/NEFT transaction environment (310,000 transaction records) to investigate why payments fail and retry, using MySQL for data modeling and 14 SQL queries (CTEs, window functions) to classify failure patterns. The analysis concludes with a data-driven retry-rule proposal and a Power BI/Excel business summary — the deliverable format a Business/Data Analyst would hand to a Payments Operations stakeholder.

The dataset is simulated to realistic industry parameters, not drawn from a live production system.

2. Business Problem

Failed digital payments are automatically retried, but the current retry logic treats every failure the same way regardless of cause. This means transactions that fail for genuinely unrecoverable reasons (invalid account, blocked beneficiary, invalid IFSC) are retried just as often as transactions that fail for recoverable, transient reasons (bank server timeout, network delay) — inflating retry volume without improving the overall success rate, adding unnecessary load to settlement systems, and giving operations teams no visibility into which failure types are actually worth retrying.

3. Project Objective

- Classify failure and retry patterns across 310,000 simulated UPI and NEFT transactions using SQL.
- Identify the smallest set of failure codes responsible for the majority of retry volume.
- Propose a data-driven retry rule that suppresses retries on non-recoverable failures.
- Estimate the retry-volume reduction the proposed rule would deliver against the historical dataset.

- Translate the technical findings into a business-facing recommendation using Power BI and Excel.

4. Scope

In scope: transaction-level failure and retry data for UPI and NEFT payment rails; failure-code classification; retry-volume and Pareto analysis; retry-rule design and simulated impact; business-facing reporting.

Out of scope: real-time fraud detection, settlement reconciliation, cross-bank dispute resolution, and any change to the underlying payment-rail protocol itself.

5. Stakeholders

Role	Stakeholder	Interest / Influence
Business Sponsor	Head of Payments Operations	Owns retry-cost and success-rate targets
Process Owner	Payments Reliability Lead	Accountable for retry policy configuration
Analyst	Pratik Kumar (Business/Data Analyst)	Performs the SQL analysis and rule design
Engineering	Payments Platform Team	Implements the retry rule in production
Customer Support	Escalations Team	Handles customer impact of failed payments
End User	Payer / Payee	Experiences transaction success or failure directly

6. Existing AS-IS Process (current retry handling)

Today, every failed transaction — regardless of its failure code — is retried up to a fixed number of times on a fixed schedule. There is no classification step that distinguishes a recoverable failure (e.g., bank server busy, gateway timeout) from a non-recoverable one

(e.g., invalid account number, blocked beneficiary, invalid IFSC code). Failure-code analysis, where it happens at all, is done manually and ad hoc rather than as a routine, data-driven process.

7. Problems in Current Process

- Blanket retry policy does not distinguish recoverable from non-recoverable failures.
- Retry volume is inflated without a corresponding improvement in success rate.
- Unnecessary retries add avoidable load to settlement and gateway systems.
- No ranked visibility into which failure codes actually drive most retries.
- Failure analysis is reactive and manual rather than a standing, repeatable process.

8. Proposed TO-BE Process (data-driven retry rule)

Each failure code is classified as recoverable or non-recoverable based on its historical retry-success pattern in the 310,000-transaction dataset. Non-recoverable codes are suppressed from retry entirely; recoverable codes retain a retry, with backoff timing informed by how quickly those failures typically resolve. Applied against the historical dataset, this rule is estimated to reduce total retry volume by 38%, with no reduction in the retries that were actually likely to succeed.

9. Business Requirements

ID	Requirement
BR-01	Classify every failed transaction by a standardized failure code
BR-02	Rank failure codes by their share of total retry volume
BR-03	Distinguish recoverable from non-recoverable failure codes
BR-04	Suppress retries on non-recoverable failure codes
BR-05	Estimate the retry-volume reduction of the proposed rule before rollout

ID	Requirement
BR-06	Present findings in business language for a Payments Operations audience

10. Functional Requirements

ID	Functional Requirement	Linked BR
FR-01	Analysis shall tag every transaction record with its failure code and retry count	BR-01
FR-02	Analysis shall compute each failure code's share of total retries using window functions	BR-02
FR-03	Analysis shall rank failure codes to identify the minimum set driving ≥60% of retries	BR-02
FR-04	Rule logic shall classify each failure code as recoverable or non-recoverable	BR-03
FR-05	Rule logic shall suppress the retry step for non-recoverable codes	BR-04
FR-06	Analysis shall simulate the proposed rule against historical data to estimate retry-volume change	BR-05
FR-07	Findings shall be exportable as a Power BI dashboard and an Excel summary	BR-06

11. Non-Functional Requirements

Category	Requirement
Performance	Queries return results over 310,000+ rows in under 10 seconds on standard MySQL indexing
Accuracy	Failure-code classification is reproducible and version-controlled, not ad hoc

Category	Requirement
Data Privacy	Transaction records contain no raw PII; account identifiers are masked/simulated
Auditability	Any change to the retry rule's logic is documented with the data supporting it
Usability	Power BI dashboard is readable by a non-technical operations stakeholder without SQL knowledge

12. User Stories

ID	User Story
US-01	As a payments analyst, I want every failed transaction tagged with a standardized failure code, so that I can classify failure patterns consistently.
US-02	As a payments analyst, I want failure codes ranked by their share of total retries, so that I can prioritize which ones matter most.
US-03	As a payments reliability lead, I want non-recoverable failures excluded from retry, so that we stop wasting retry attempts that can never succeed.
US-04	As a payments ops lead, I want the proposed retry rule simulated against historical data, so that I can estimate its impact before rollout.
US-05	As a non-technical stakeholder, I want the findings in a Power BI dashboard, so that I can act on them without reading SQL.

13. Acceptance Criteria

US-02 — Rank failure codes by retry share

- Given the full 310,000-transaction dataset, when retries are aggregated by failure code using a window function, then codes are ranked by percentage of total retries, descending.
- Given the ranked list, when the top codes are summed, then the analysis identifies the minimum set of codes accounting for ≥60% of retries.

US-03 — Suppress non-recoverable retries

- Given a transaction fails with a code classified as non-recoverable, when the retry step runs, then no retry is attempted and the transaction is marked final-failed.
- Given a transaction fails with a recoverable code, when the retry step runs, then the existing retry logic applies unchanged.

US-04 — Simulate rule impact

- Given the historical dataset, when the proposed rule is applied retroactively, then the analysis outputs the resulting retry volume and the percentage change versus baseline.

14. Business Rules

ID	Rule
BRule-01	Failure codes indicating a permanently invalid destination (invalid account, blocked beneficiary, invalid IFSC) are classified non-recoverable and never retried.
BRule-02	Failure codes indicating a transient system condition (gateway timeout, bank server busy) are classified recoverable and retried per existing backoff logic.
BRule-03	A failure code's classification is reviewed if its recoverable/non-recoverable outcome pattern shifts materially in a later data refresh.
BRule-04	The top failure codes by retry share are reviewed monthly to catch new dominant failure patterns early.

15. Process Flow / Analysis Methodology

1. Loaded 310,000 simulated UPI and NEFT transaction records into MySQL.
2. Built 14 SQL queries using CTEs and window functions to tag, aggregate, and rank transactions by failure code and retry count.
3. Identified that 3 failure codes account for 61% of all retries — a Pareto-style concentration.
4. Classified each failure code as recoverable or non-recoverable based on its historical retry-success pattern.
5. Simulated the proposed retry rule against the historical dataset, estimating a 38% reduction in retry volume.
6. Converted the findings into a Power BI dashboard and an Excel business summary for a non-technical audience.

16. Assumptions

- The simulated dataset's failure-code distribution approximates real-world UPI/NEFT retry patterns closely enough to be directionally reliable.
- Failure-code definitions follow standard NPCI/bank error-code taxonomy.
- The 38% retry-volume reduction is a simulation-based estimate, not the result of a live A/B test, and would need validation in production before being reported as an audited figure.
- Retry backoff timing for recoverable codes is unchanged by this proposal; only the non-recoverable suppression is new.

17. Risks

Risk	Likelihood	Impact	Mitigation
Simulated data distribution diverges from live production patterns	Medium	High	Validate the rule against a production sample before full rollout
Suppressing retries on a misclassified code reduces recoverable successes	Low	High	Shadow-mode the rule, comparing outcomes before enforcing suppression

Risk	Likelihood	Impact	Mitigation
Failure-code taxonomy drifts as payment rails evolve	Low	Medium	Monthly review of top failure codes (BRule-04)
Stakeholders over-read the 38% estimate as an audited result	Medium	Medium	Clearly label it as a simulation-based estimate in all reporting

18. Success Metrics

Metric	Baseline	Target
Transactions analyzed	—	310,000
SQL queries built	—	14
Failure codes driving majority of retries	Unranked	3 codes = 61% of retries
Estimated retry-volume reduction	0% (current blanket retry)	38%
Reporting format	Ad hoc / manual	Power BI dashboard + Excel summary